

-:TRAIN PLATFORM MANAGEMENT SYSTEM:-

Using Greedy Algorithm

Project Report Submitted For Design and Analysis of Algorithms

TEAM MEMBER:

- 1) KUNAL DAS - 27631725018**
- 2) SIMI MALLICK - 27631725022**
- 3) PRANJAL DAS SARMA - 27631724005**
- 4) CHANDAN PAL - 27631725023**
- 5) DEBMALYA GURIA - 27631724004**

GUIDE/ FACULTY : MS. KAIFA SULTANA.

SEMESTER : 4TH SEM.

DEPERTMENT : CSE-CS.

ACADAMIC YEAR : 2025-2026.

DATE :

Sl. No. **Contents**

Page No.

<u>1</u>	<u>Problem Statement</u>
<u>2</u>	<u>Application Area</u>
<u>3</u>	<u>Flow Diagram</u>
<u>4</u>	<u>Algorithm</u>
<u>5</u>	<u>Source Code</u>
<u>6</u>	<u>Input and Output</u>
<u>7</u>	<u>GitHub Project Link</u>
<u>8</u>	<u>Time Complexity</u>
<u>9</u>	<u>Acknowledgement</u>
<u>10</u>	<u>Conclusion</u>
<u>11</u>	<u>References</u>

1. Problem Statement

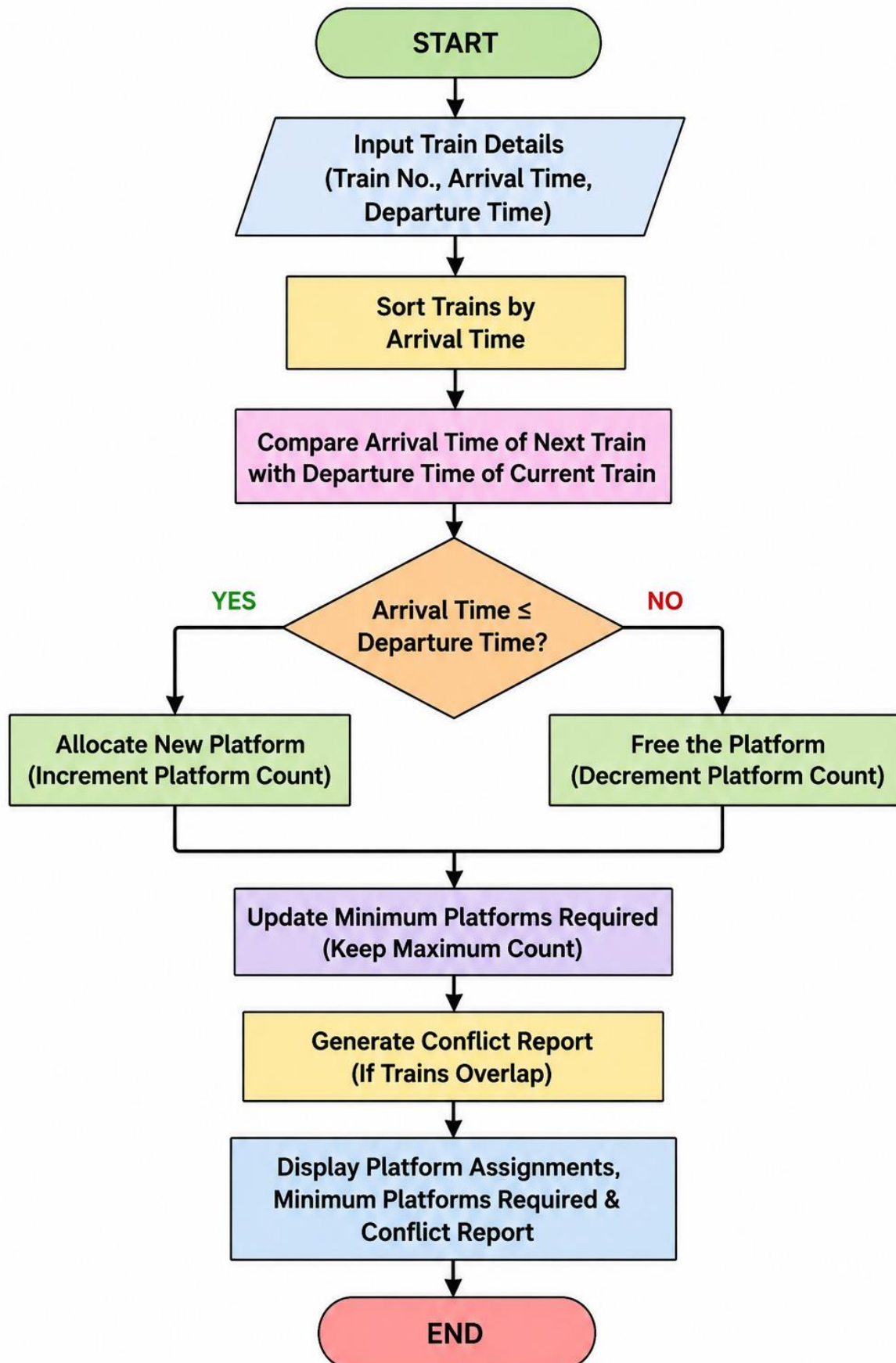
Railway stations often face scheduling conflicts when multiple trains arrive and depart around the same time. The objective of this project is to determine the minimum number of platforms required so that no train has to wait.

The system takes train arrival and departure times as input and uses a Greedy Algorithm to assign platforms efficiently while minimizing platform requirements.

2. Application Area

- Railway Management Systems
- Airport Gate Scheduling
- Bus Terminal Scheduling
- Event Scheduling Systems
- Resource Allocation Problems
- Traffic Management Systems

3. Flow Diagram:



4. Algorithm:

Greedy Algorithm for Platform Allocation

1. Read arrival and departure times of all trains.
2. Sort arrival times and departure times separately.
3. Initialize:
 - platform_needed = 1
 - result = 1
4. Compare arrival and departure times.
5. If arrival time is earlier than next departure:
 - Increase platform count.
6. Otherwise:
 - Decrease platform count.
7. Update maximum platforms used.
8. Repeat until all trains are processed.
9. Display minimum platforms required.

5. Source Code (C++):

```
#include <stdio.h>
```

```
void sort(int arr[], int n)
```

```
{
```

```
    int i, j, temp;
```

```
    for(i = 0; i < n - 1; i++)
```

```
    {
```

```
        for(j = 0; j < n - i - 1; j++)
```

```
        {
```

```
            if(arr[j] > arr[j + 1])
```

```
            {
```

```
                temp = arr[j];
```

```
                arr[j] = arr[j + 1];
```

```
                arr[j + 1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
int findPlatform(int arr[], int dep[], int n)
```

```
{
```

```
    sort(arr, n);
```

```
    sort(dep, n);
```

```
int plat_needed = 1;
int result = 1;
int i = 1, j = 0;

while(i < n && j < n)
{
    if(arr[i] <= dep[j])
    {
        plat_needed++;
        i++;
    }
    else
    {
        plat_needed--;
        j++;
    }

    if(plat_needed > result)
        result = plat_needed;
}

return result;
}

int main()
{
```

```
int arr[] = {900, 910, 940};  
int dep[] = {930, 1000, 1030};  
  
int n = 3;  
  
printf("Minimum Platforms Required = %d\n",  
    findPlatform(arr, dep, n));  
  
return 0;  
}
```


6. Input and Output:

Input

Train Arrival Departure

T1 09:00 09:30

T2 09:10 10:00

T3 09:40 10:30

Output

Minimum Platforms Required = 2

Train

Platform

T1

P1

T2

P2

T3

P1

Conflict Detected:

T1 and T2 overlap between 09:10 and 09:30

No waiting train because Platform 2 is available.

7. GitHub Project Link

After uploading your code to GitHub, add:

GitHub Repository:

<https://github.com/yourusername/train-platform-management-system>

8. Time Complexity

Sorting

- Arrival Times Sorting $\rightarrow O(n \log n)$
- Departure Times Sorting $\rightarrow O(n \log n)$

Platform Calculation

- Single Traversal $\rightarrow O(n)$

Total Time Complexity

$O(n \log n)$

Space Complexity

$O(1)$ (excluding input storage)

9. Acknowledgement

I would like to express my sincere gratitude to our faculty members for their valuable guidance and support throughout this project. Their suggestions helped me understand the practical application of Greedy Algorithms in solving real-world scheduling problems.

I also thank my college for providing the necessary resources and facilities for completing this project successfully.

Finally, I thank my classmates and friends for their encouragement and assistance during the development of this project.

10. Conclusion

The Train Platform Management System successfully determines the minimum number of platforms required at a railway station using the Greedy Algorithm. By sorting train arrival and departure times and efficiently allocating platforms, scheduling conflicts can be minimized and train delays can be avoided.

The project demonstrates the practical use of Greedy Algorithms in real-world resource allocation and scheduling problems. The obtained time complexity of $O(n \log n)$ makes the solution efficient even for large numbers of trains.

11. References

1. Introduction to Algorithms – Cormen, Leiserson, Rivest, and Stein.
2. Data Structures and Algorithms in C++ – Goodrich.
3. GeeksforGeeks – Platform Allocation Problem.
4. Railway Scheduling Research Papers.
5. C++ Documentation and STL Library Reference.